Plan de trabajo del equipo

Versión simple · Go + React + PostgreSQL

Plataforma de Seguimiento de Células de Bosque · UCC Sostenible
Versión 4 · 18 de octubre de 2026 · Reemplaza a las versiones anteriores

Qué cambió respecto del plan anterior

El backend se hace en Go, el frontend en React y la base en PostgreSQL. Además, ya no hay una persona de "datos", otra de "campo" y otra de "interfaz": **cada una hace funcionalidades completas**, con su parte de Go y su parte de React. Así los tres aprenden lo mismo, cualquiera puede ayudar a cualquiera y nadie queda esperando que otro termine "su capa". Los tickets bajaron de 56 a 27 y cada uno trae una lista de pasos para ir tachando. En esta versión se suma el T28.

Novedad de esta versión (16/10): la clienta respondió las preguntas abiertas. La altura y la cantidad de flores van en **desplegable**; **no se registran diámetro ni polinizadores**; un individuo muerto queda en gris y no se le carga nada más, salvo que se lo reponga con el botón "Cargar uno nuevo"; las nuevas plántulas son un campo Sí/No y la carga puede llevar más de 2 minutos. Además, la tercera forma de vida se llama **Gramínea** y no Herbácea: hay que cambiarlo en todo el proyecto (ticket nuevo **T28**). Se actualizaron T02, T08, T13, T14, T18, T21, T23, T25, el calendario y el ejemplo de la API. El detalle está en el documento de requerimientos v5 (secciones 2, 4, 6, 12 y 15, CAR-18 y CAR-20).

Cómo leer este plan

Si no tienen tiempo de leer todo, lean en este orden: la sección 1 (palabras que van a aparecer), la sección 3 (cómo nos dividimos), la sección 5 (la primera semana) y la sección 6 (el calendario). Los tickets de la sección 8 los lee cada uno cuando le toca.

Contenido

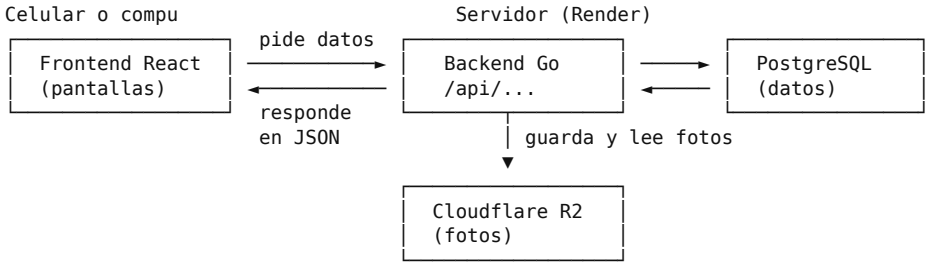
1. Palabras que van a aparecer · 2. Cómo está armada la app · 3. Cómo nos dividimos · 4. Reglas para no pisarse · 5. Semana 0: arrancamos juntos · 6. Calendario de las 6 semanas · 7. La API (lista de endpoints) · 8. Tickets paso a paso · 9. Qué hacer si te trabás · 10. Lo que queda para después

1. Palabras que van a aparecer

Palabra	Qué significa en este proyecto
Frontend	Lo que se ve en el celular o la compu: pantallas, botones, formularios. Lo hacemos en React.
Backend	El programa que corre en el servidor. Recibe pedidos del frontend, controla permisos, lee y guarda en la base. Lo hacemos en Go.
Base de datos	Donde quedan guardados los datos (usuarios, células, registros). Usamos PostgreSQL.
API	El "menú" de pedidos que el frontend le puede hacer al backend. Está en la sección 7.
Endpoint	Un pedido concreto de la API. Ej.: `GET /api/celulas/45` = "dame los datos de la célula 45".
GET / POST / PUT	GET pide datos, POST crea algo nuevo, PUT modifica algo que ya existe.
JSON	El formato de texto en el que viajan los datos entre frontend y backend.
Migración	Un archivo SQL numerado que crea o cambia tablas. Todos corren las mismas migraciones y así tienen la misma base.
Seed	Un script que llena la base con datos de prueba, para no trabajar con pantallas vacías.
Mock (datos simulados)	Datos falsos escritos a mano en el frontend para poder hacer una pantalla antes de que su endpoint exista.
Middleware	Código de Go que se ejecuta antes de cada endpoint. Lo usamos para chequear que el usuario inició sesión y tiene permiso.
Sesión	Cómo el backend recuerda que ya iniciaste sesión: guarda un código secreto en una cookie del navegador.
Rama (branch)	Una copia del código donde trabajás tu ticket sin romper lo de los demás.
Pull Request (PR)	El pedido para sumar tu rama al código principal. Otra persona lo revisa antes de aceptarlo.
main	La rama principal. Siempre tiene que funcionar, porque lo que está en main se publica solo.
Deploy	Publicar la app en internet para que se pueda usar desde cualquier celular.
Ticket	Una tarea chica y concreta, con un responsable y una lista de pasos. Están en la sección 8.
Sprint / semana	Cada semana es un sprint: al final de la semana, lo planeado tiene que funcionar.

2. Cómo está armada la app

La app tiene cuatro partes. El usuario solo ve el frontend; todo lo demás pasa por detrás.



Para simplificar, **el mismo programa de Go también entrega las pantallas de React**. Así todo vive en una sola dirección web, no hay que configurar permisos entre dominios (CORS) y hay un solo deploy.

Herramientas

Parte	Herramienta	Para qué
Frontend	React + Vite, en JavaScript	Las pantallas. Si el equipo ya sabe TypeScript, úsenlo; si no, JavaScript alcanza.
	React Router	Pasar de una pantalla a otra.
	Tailwind CSS	Estilos. Los colores y tipografías salen del DESIGN.md de Stitch.
	Recharts	Gráficos.
	browser-image-compression · qrcode	Achicar las fotos antes de subirlas · generar el QR.
Backend	Go 1.22 o más nuevo, con net/http	El servidor. La librería estándar de Go ya permite rutas como `GET /api/celulas/{id}`, no hace falta un framework.
	pgx	Conectarse a PostgreSQL y hacer consultas SQL.
	goose	Correr las migraciones.
	bcrypt (golang.org/x/crypto)	Guardar las contraseñas de forma segura.
	excelize	Generar el Excel de la tabla de datos.
	aws-sdk-go-v2 (S3)	Subir y leer fotos en Cloudflare R2, que funciona igual que S3.
Base de datos	PostgreSQL 16	Datos relacionados con reglas claras (un registro por célula por mes, un número por célula), métricas con SQL y buen soporte en Go.
En tu compu	Docker Desktop	Levantar PostgreSQL con un solo comando, igual para los tres.
Publicación	Render (Go + PostgreSQL) y Cloudflare R2 (fotos)	Pocas cuentas y deploy automático. Ojo con el plan gratuito de Render: la base se borra a los 30 días de crearse y no tiene backups, y el servicio se duerme a los 15 minutos sin tráfico. Acuerden con UCC quién paga (pregunta 13 del v5).

Carpetas del repositorio

```
celulas-bosque/
├── backend/
│   ├── main.go            ← arranca el servidor y registra TODAS las rutas
│   ├── migrations/        ← archivos .sql de la base (goose)
│   ├── seed/seed.sql      ← datos de prueba
│   └── internal/
│       ├── db/            ← conexión a la base
│       ├── auth/          ← login, sesiones y permisos (middleware)
│       ├── respuestas/    ← funciones para responder JSON y errores
│       ├── estados/       ← EstadoCelula y PeriodoActual (T11)
│       ├── almacen/       ← guardar y leer fotos en Cloudflare R2 (T13)
│       └── handlers/      ← un archivo por funcionalidad:
│           ├── lugares.go   celulas.go   registros.go   fotos.go
│           ├── metricas.go  usuarios.go  admin.go       datos.go
│           └── publico.go
└── frontend/
    └── src/
        ├── App.jsx        ← registra TODAS las pantallas (rutas)
        ├── api.js         ← funciones para llamar al backend
        ├── components/    ← piezas compartidas (Boton, Tarjeta, Plano, Foto...)
        └── pages/         ← una carpeta por pantalla:
            Login/  Inicio/  Celulas/  Isleta/  Carga/  NuevaCelula/  Ficha/
            Resultados/  Datos/  Usuarios/  Lugares/  Publico/
```

3. Cómo nos dividimos

Somos tres: Persona A, Persona B y Persona C (pongan los nombres en la primera reunión). Cada semana, cada persona toma una funcionalidad completa: hace su endpoint en Go, su pantalla en React y la prueba en el celular. Las funcionalidades están elegidas para que cada uno toque casi siempre archivos distintos (ver sección 4).

Todos hacen Go y React todas las semanas. Lo que cambia es el tema: así, al final del proyecto los tres conocen toda la app, y si alguien se enferma o se traba, cualquiera puede seguir su ticket.

Semana	Persona A	Persona B	Persona C
0 · 30/9 – 4/10	Repo, base de datos y deploy	Esqueleto del servidor Go, preguntas a la clienta y diseño en Stitch	Esqueleto de React y componentes base
1 · 5/10 – 11/10	Login y permisos	Inicio y explorar células	Plano de la isleta
2 · 12/10 – 18/10	Fotos y guardar registro	Pantalla de carga mensual	Ficha de la célula
3 · 19/10 – 25/10	Usuarios	Nueva célula, editar registro y borrador (y el cambio a Gramínea, T28)	Resultados y métricas
4 · 26/10 – 1/11	Lugares, isletas y planos + backups	Tabla de datos y Excel	Página pública y QR
5 · 2/11 – 8/11	Documento de traspaso	Manual para cargadores	Demo de la plantación

En las semanas 3, 4 y 5 hay además tareas de todos juntos: la versión de prueba para la clienta, la prueba en el campo, la carga de datos reales y las correcciones.

Quién revisa a quién

Para que todos lean el código de todos, las revisiones rotan: A revisa las PR de B, B revisa las de C y C revisa las de A. Revisar no es solo aprobar: bajá la rama, probala y preguntá lo que no entiendas.

Trabajo en pareja

Hay dos momentos donde conviene trabajar de a dos, compartiendo pantalla: el login y los permisos (semana 1, A con quien pueda) y la carga mensual (semana 2, A y B, porque la pantalla de B usa los endpoints de A). Son las partes más difíciles y las más importantes.

4. Reglas para no pisarse

#	Regla	Por qué
1	Cada ticket tiene sus archivos. Trabajá en el archivo de tu funcionalidad: `handlers/celulas.go` y `pages/Celulas/`, por ejemplo.	Si cada uno toca archivos distintos, Git junta todo solo, sin conflictos.
2	Hay tres archivos compartidos: `main.go` (rutas del backend), `App.jsx` (rutas del frontend) y `api.js`. En esos solo se agregan líneas, nunca se borran las de otro, y la PR se hace ese mismo día.	Son los únicos lugares donde los tres escriben.
3	Todas las tablas se crean en la semana 0. Si después necesitás cambiar la base, creá una migración nueva y avisá en el grupo antes de hacerla.	Dos cambios de base al mismo tiempo pueden romper la base de los otros.

#	Regla	Por qué
4	Nunca se sube nada directo a main. Rama por ticket (`t14-carga-mensual`), PR, revisión de otra persona, y recién ahí se une a main.	Main se publica solo; si se rompe, se rompe para la clienta.
5	Si el endpoint de otro no está listo, usá un mock. Escribí en tu pantalla una respuesta falsa con el mismo formato de la sección 7 y seguí. Cuando el endpoint esté, cambiás el mock por la llamada real.	Así nadie espera a nadie.
6	Todas las mañanas, antes de empezar: `git pull` de main y actualizá tu rama.	Cuanto más seguido se junta el código, más chicos son los conflictos.
7	Reunión de 10 minutos por día (o un mensaje en el grupo): qué hice ayer, qué hago hoy, en qué estoy trabado.	Un problema dicho a tiempo se resuelve en 10 minutos; callado, puede costar un día.

Tablero

Usen GitHub Projects (queda junto al código) con cinco columnas: Por hacer → Haciendo → En revisión → Hecho, más Esperando a la clienta. Cada ticket de la sección 8 es una tarjeta. Regla: nadie tiene más de un ticket en "Haciendo".

5. Semana 0: arrancamos juntos

Esta semana arma la base para que desde el lunes 5 los tres puedan trabajar solos. Cada uno arma una parte distinta, pero al final de cada día le muestra a los otros dos lo que hizo en 15 minutos, así todos entienden cómo funciona cada pieza.

Día	Persona A	Persona B	Persona C
Mar 30/9	**Los tres juntos (2 h):** leer las secciones 1 a 4 de este plan, poner nombres a A, B y C, crear el tablero y hacer T01 (instalar todo). Después: **T02**, crear el repo y Docker con PostgreSQL.	Reunión y T01. Después: **T08**, mandar hoy las preguntas a la clienta.	Reunión y T01. Después: **T04**, crear el proyecto React con Vite.
Mié 1/10	T02: primera migración con todas las tablas.	**T03**: servidor Go con `/api/health` y conexión a la base.	T04: Tailwind con los colores de Stitch, rutas y barra de navegación.
Jue 2/10	T02: terminar migraciones y el README "cómo levantar el proyecto". **Los tres (1 h): T05**, repasar juntos la lista de endpoints.	T03: funciones para responder JSON y errores. T05 con los demás.	T04: componentes base. T05 con los demás.
Vie 3/10	**T07**: deploy en Render.	**T06** con C: datos de prueba (seed).	T06 con B. Terminar componentes base.
Sáb 4/10	Terminar deploy. Revisar PRs.	**T09**: pedir a Stitch las pantallas faltantes.	Revisar PRs. Probar todo desde el celular.

La semana 0 está terminada cuando…

…los tres pueden hacer esto en su propia compu: levantar la base con Docker, correr las migraciones y el seed, prender el backend y el frontend, entrar a la app y navegar entre pantallas vacías. Además, la app está publicada en Render, las preguntas a la clienta están enviadas y el diseño completo está en el repo. Si algo falta, se termina el lunes 5 antes de arrancar la semana 1.

6. Calendario de las 6 semanas

Semana	Persona A	Persona B	Persona C	Todos	Al terminar la semana se puede…
0	T02, T07	T03, T08, T09	T04	T01, T05, T06	Navegar la app publicada con datos de prueba.
1	T10	T11	T12	—	Iniciar sesión, ver el inicio, explorar células y abrir el plano de una isleta.
2	T13	T14	T15	—	Cargar el registro del mes con foto y verlo en la ficha.
3	T16	T17, T28	T18	T19 (jueves 22 o viernes 23)	**Mostrarle a la clienta una versión de prueba.**
4	T20	T21	T22	T23	Usar la app en el campo con datos reales y escanear el QR.
5	T27	T25	T26	T24	Presentar la app en la plantación y dejarla en manos de UCC.

Fechas que dependen de la clienta: las variables llegaron el 2/10 y las respuestas a las preguntas 14 a 18 del documento v5 (desplegable de altura, flores, plántulas, individuo muerto) llegaron el 16/10. Sigue abierta la pregunta 19 (cómo se mide el crecimiento de un individuo repuesto). Las métricas que quiere ver, antes del 19/10. Los planos de las isletas y la lista de usuarios, antes del 26/10. Si no llegan, se usan los valores propuestos del documento de requerimientos v5 y se ajusta después.

Estado al 18/10: el PR #16 (T13) pasó la revisión y se mergeó, y T14 arranca ahora. Como la semana 2 termina el 18/10, T14 se corre a la semana 3 (propuesta). Si T14 no está completa para la demo del 22 o 23, se muestra lo que haya y se anota qué falta. El cambio a Gramínea (T28) se hace antes de armar los bloques de T14, para no escribir código nuevo con el nombre viejo.

Si la plantación es a fin de octubre, la semana 4 pasa a ser la de lanzamiento: se hacen primero la página pública con QR (T22), la carga de datos reales y la prueba en campo (T23), y lo demás se corre.

7. La API: lista de endpoints

Esta es la lista de pedidos que el frontend le hace al backend. Se repasa entre los tres el jueves de la semana 0 (T05) y después se respeta: si alguien necesita cambiar un endpoint, avisa antes, porque otra pantalla puede estar usándolo. Todas las respuestas son JSON.

Método y ruta	Qué hace	Quién puede	Ticket
`POST /api/login`	Inicia sesión con email y contraseña.	Cualquiera	T10
`POST /api/logout`	Cierra la sesión.	Con sesión	T10
`GET /api/yo`	Devuelve el usuario actual (nombre, rol, lugar).	Con sesión	T10
`GET /api/lugares`	Lugares con cantidad de células, isletas y avance del mes.	Con sesión	T11
`GET /api/lugares/{id}/celulas`	Células de un lugar. Filtros: `?isleta=1&estado=pendiente&q=45`.	Con sesión	T11
`GET /api/isletas/{id}`	Isleta con plano, filas, columnas y cada célula con su posición y estado.	Con sesión	T12
`GET /api/celulas/{id}`	Ficha: datos, 4 individuos y todos sus registros.	Con sesión	T15
`GET /api/celulas/{id}/ultimo-registro`	El registro del mes en curso (si ya hay) y el último de un mes anterior, para precargar la carga mensual.	Con sesión	T14
`POST /api/celulas`	Crea una célula y sus 4 individuos.	Admin; cargador en su lugar	T17
`POST /api/fotos`	Sube una foto (campo `foto`), le borra los metadatos y devuelve su id.	Admin; cargador	T13
`GET /api/fotos/{id}`	Devuelve la imagen si el usuario tiene permiso.	Según el lugar	T13
`POST /api/celulas/{id}/registros`	Guarda el registro del mes. Responde 409 con `registroId` si ya hay uno de ese mes.	Admin; cargador en su lugar	T13
`PUT /api/registros/{id}`	Modifica un registro. El cargador solo edita el del mes actual de su lugar.	Admin; cargador en su lugar	T13
`GET /api/metricas`	Métricas y datos de gráficos. Filtros: lugar, isleta, desde, hasta, forma de vida.	Con sesión	T18
`GET /api/datos`	Tabla de datos paginada, con los mismos filtros.	Con sesión	T21
`GET /api/datos/excel`	Descarga el Excel con los mismos filtros.	Con sesión	T21
`GET · POST · PUT /api/usuarios`	Listar, crear y modificar usuarios.	Admin	T16
`GET · POST · PUT /api/admin/lugares`	Listar, crear y modificar lugares.	Admin	T20
`GET · POST · PUT /api/admin/isletas`	Listar, crear y modificar isletas.	Admin	T20
`POST /api/admin/isletas/{id}/plano`	Sube la imagen del plano.	Admin	T20
`POST /api/admin/isletas/{id}/celulas-lote`	Crea varias células de una vez.	Admin	T20

Método y ruta	Qué hace	Quién puede	Ticket
`GET /api/publico/{letra}`	Resultados y células de un lugar público.	Cualquiera, sin sesión	T22

Respuestas de error

Todos los endpoints responden los errores igual, así el frontend los muestra de una sola forma:

```
{ "error": "La altura del Arbusto tiene que estar entre 1 y 999 cm" }
```

`400` = datos inválidos · `401` = no inició sesión · `403` = no tiene permiso · `404` = no existe · `409` = ya existe (ej. registro de ese mes) · `500` = error del servidor

Un 409 al guardar un registro trae además el id del que ya existe, para abrirlo en modo edición: `{ "error": "Esta célula ya tiene un registro de ese mes", "registroId": 812 }`

Ejemplo: guardar un registro

Un individuo muerto se manda solo con `formaDeVida` y `vivo: false`. Diámetro y polinizadores ya no existen (descartados el 16/10).

```
POST /api/celulas/45/registros
{
  "fecha": "2026-11-18",
  "fotoId": 812,
  "observaciones": "Hojas nuevas en la base del arbusto",
  "individuos": [
    { "formaDeVida": 1, "vivo": true, "alturaCm": 185,
      "flores": 0, "frutos": true, "plantulasNuevas": false },
    { "formaDeVida": 2, "vivo": true, "alturaCm": 94,
      "flores": 12, "frutos": false, "plantulasNuevas": true },
    { "formaDeVida": 3, "vivo": false },
    { "formaDeVida": 4, "vivo": true, "alturaCm": 18,
      "flores": 4, "frutos": false, "plantulasNuevas": false }
  ]
}
→ { "id": 1023 }
```

Ejemplo: precargar la carga mensual (propuesta de T14)

```
GET /api/celulas/45/ultimo-registro
{
  "celula": { "id": 45, "codigo": "A-1-45", "lugarId": 1, "lugar": "Campus UCC",
              "isleta": 1, "fechaPlantacion": "2026-10-20", "dadaDeBaja": false },
  "periodoActual": "2026-11",
  "actual": null,            // registro del mes en curso, o null si todavía no hay
  "anterior": {              // último registro de un mes anterior, o null
    "id": 812, "fecha": "2026-10-18", "periodo": "2026-10", "fotoId": 55,
    "observaciones": null,
    "individuos": [
      { "formaDeVida": 1, "vivo": true, "alturaCm": 185, "flores": 0,
        "frutos": true, "plantulasNuevas": false },
      { "formaDeVida": 3, "vivo": false }
    ]
  }
}
```

8. Tickets paso a paso

Cada ticket tiene una lista de pasos para ir tachando. "Está listo cuando" es lo que hay que poder mostrar para moverlo a Hecho. Los códigos de requerimientos (AUT-01, CAR-06…) son los del documento de requerimientos v5: ahí están los detalles si algo no queda claro.

Semana 0 · Arranque

T01 · Instalar herramientas	Todos · Semana 0
Qué es	Que los tres tengan lo mismo instalado y puedan correr el proyecto.
Pasos	☐ Instalar Git, Go (1.22 o más), Node.js (LTS), Docker Desktop y VS Code con las extensiones de Go y ESLint. ☐ Crear cuenta de GitHub si no tienen y aceptar la invitación al repo. ☐ Hacer el tour oficial de Go (go.dev/tour) hasta "Methods" si nunca usaron Go.
Está listo cuando	Los tres corren `go version`, `node -v` y `docker run hello-world` sin errores.

T02 · Repositorio y base de datos	Persona A · Semana 0
Qué es	Crear el repo con las carpetas de la sección 2 y la base con todas las tablas.
Repo y base	☐ Crear el repo en GitHub con las carpetas backend y frontend y el .gitignore. ☐ Crear docker-compose.yml con PostgreSQL 16. ☐ Instalar goose y crear las migraciones de todas las tablas (sección 11 del documento v5): lugares, isletas, células, formas de vida, individuos, variables, registros, mediciones, usuarios, sesiones y cambios. ☐ Cargar el catálogo de formas de vida (1 Árbol, 2 Arbusto, 3 Gramínea, 4 Rastrera) y el de variables por individuo: vivo/muerto, altura (cm), cantidad de flores (0 a 100), presencia de frutos (sí/no) y nuevas plántulas (sí/no), para las 4 formas de vida. ☐ Agregar las restricciones: letra única, número de isleta único por lugar, número y posición de célula únicos por isleta, un registro por célula por período. ☐ Escribir el README con los pasos para levantar todo.
Está listo cuando	Otra persona clona el repo, sigue el README y tiene la base con todas las tablas en menos de 15 minutos.
Requerimientos	RN-01 a RN-04

T03 · Esqueleto del servidor Go	Persona B · Semana 0
Qué es	Un servidor Go que arranca, se conecta a la base y responde.
Backend (Go)	☐ Crear main.go con net/http y la ruta `GET /api/health` que responde `{"ok": true}`. ☐ Conectarse a PostgreSQL con pgx leyendo la dirección desde una variable de entorno. ☐ Crear el paquete respuestas con dos funciones: `JSON(w, datos)` y `Error(w, codigo, mensaje)`. ☐ Hacer que el servidor entregue los archivos de `frontend/dist` para cualquier ruta que no empiece con /api. ☐ Configurar el proxy de Vite para que en desarrollo /api vaya al servidor Go.
Está listo cuando	Con el backend y el frontend prendidos, el frontend llama a /api/health y muestra la respuesta.

T04 · Esqueleto de React y componentes base	Persona C · Semana 0
Qué es	El proyecto React con el estilo de Stitch y las piezas que todos van a reutilizar.
Frontend (React)	☐ Crear el proyecto con Vite e instalar React Router y Tailwind. ☐ Pasar los colores, tipografía y bordes del DESIGN.md a la configuración de Tailwind. ☐ Crear las rutas de todas las pantallas (vacías) en App.jsx y la barra inferior de navegación. ☐ Crear en components/: Boton, Tarjeta, ChipEstado, ContadorMasMenos (−1/+1), BotonesVivoMuerto, BotonesSiNo, SelectorNumero, Modal, PanelInferior, Cargando, Vacio y MensajeError. ☐ Crear api.js con una función pedir(ruta, opciones) que maneje JSON y errores.
Está listo cuando	Se puede navegar entre las pantallas vacías y hay una página de prueba que muestra todos los componentes.
Requerimientos	RNF-USA-01 a 07

T05 · Repasar juntos la API	Todos · Jueves 2/10 · 1 hora
Qué es	Leer juntos la sección 7, cambiar lo que no les cierre y dejarla fija.
Está listo cuando	Los tres están de acuerdo con la lista de endpoints y el formato de los datos.

T06 · Datos de prueba	Personas B y C · Semana 0
Qué es	Un script que llena la base con datos parecidos a los reales.
Pasos	☐ Escribir seed/seed.sql con 3 lugares (A Campus UCC, B Alta Gracia, C Despeñaderos), 5 isletas con plano, 120 células con sus 4 individuos, 3 meses de registros (con altura, flores, frutos y plántulas variados) y un usuario de cada rol con contraseña conocida. ☐ Subir 10 fotos de muestra y usarlas en los registros.
Está listo cuando	Cualquiera puede borrar la base, correr migraciones y seed y tener datos para trabajar.

T07 · Publicar la app	Persona A · Semana 0
Qué es	Que la app esté en internet y se actualice sola.
Pasos	☐ Crear el proyecto en Render con un Web Service para Go y una base PostgreSQL. Anotar qué plan es y en qué fecha se creó la base (la gratuita vence a los 30 días). ☐ Configurar que cada cambio en main compile el frontend, compile Go y publique. ☐ Correr las migraciones en producción. ☐ Crear la cuenta de Cloudflare R2 y el bucket para fotos (se usa desde la semana 2).
Está listo cuando	La dirección pública abre la app desde un celular.

T08 · Preguntas a la clienta	Persona B · Martes 30/9
Qué es	Mandar las preguntas de la sección 15 del documento v5.
Está listo cuando	Las preguntas están enviadas y las respuestas se anotan en el repo a medida que llegan.
Estado (16/10)	Llegaron las respuestas a las preguntas 4 y 14 a 18. Sigue abierta la pregunta 19 (crecimiento de un individuo repuesto) y las demás de la sección 15.
Requerimientos	Sección 15 del documento v5

T09 · Completar el diseño en Stitch	Persona B · Semana 0
Qué es	Pedir las pantallas que faltan con los prompts ya armados y limpiar las existentes. Incluye cambiar Herbácea por Gramínea en las pantallas.
Está listo cuando	El .zip con todas las pantallas está en el repo, en la carpeta diseno/.
Requerimientos	Sección 13 del documento v5

Semana 1 · Entrar y ver

T10 · Login y permisos	Persona A (en pareja) · Semana 1
Qué es	Iniciar sesión y que cada endpoint sepa quién es el usuario y qué puede hacer.
Backend (Go)	☐ Endpoint `POST /api/login`: busca el usuario por email, compara la contraseña con bcrypt, crea una sesión con un código al azar y lo guarda en una cookie HttpOnly. ☐ Endpoints `POST /api/logout` y `GET /api/yo`. ☐ Middleware RequiereSesion: si no hay sesión válida, responde 401. ☐ Funciones EsAdmin(usuario) y PuedeEditarLugar(usuario, lugarID): admin puede todo, cargador solo su lugar, lector nada. ☐ Usuario deshabilitado: responde 403 con el mensaje de contacto.
Frontend (React)	☐ Pantalla de login con mostrar contraseña, mensajes de error y botón "Ver resultados del Campus UCC". ☐ Guardar el usuario actual (de /api/yo) en un contexto de React. ☐ Si no hay sesión, cualquier pantalla interna manda al login. ☐ Botón cerrar sesión en "Más".
Está listo cuando	Los tres usuarios del seed entran y ven su rol; con contraseña incorrecta aparece el error; sin sesión no se puede ver nada interno.
Requerimientos	AUT-01, 02, 03, 06, 07, 09 · RNF-SEG-01, 03

T11 · Inicio y explorar células	Persona B · Semana 1
Qué es	La pantalla de inicio y el listado de células con filtros.
Backend (Go)	☐ Función EstadoCelula (al día, pendiente, sin registro inicial, dada de baja), según la sección 5 del documento v5. Subirla a main el primer día, porque C también la usa. ☐ Endpoint `GET /api/lugares` con cantidad de células, isletas y avance del mes. ☐ Endpoint `GET /api/lugares/{id}/celulas` con filtros de isleta, estado y búsqueda por código.
Frontend (React)	☐ Pantalla Inicio: saludo, avance del mes, accesos rápidos (solo admin y cargador) y tarjetas de lugares. ☐ Pantalla Células: selector de lugar, filtros, buscador y tarjetas con el estado de los 4 individuos. ☐ Botón "Cargar registro" solo si PuedeEditarLugar; si no, aviso de solo lectura.
Está listo cuando	Con el usuario cargador se ve su lugar primero y el botón de carga solo en ese lugar; el lector no ve botones de carga.
Requerimientos	INI-01 a 07 · EXP-01 a 05

T12 · Plano de la isleta	Persona C · Semana 1
Qué es	El plano de dron con la grilla para encontrar la célula en el campo.
Backend (Go)	☐ Endpoint `GET /api/isletas/{id}` con el plano, filas, columnas y cada célula con su posición y estado (usa EstadoCelula de B; mientras tanto, mock).
Frontend (React)	☐ Componente Plano en components/: imagen con grilla encima, número en cada celda, colores por estado, leyenda y zoom con dos dedos. Lo van a reutilizar B y A, así que tiene que recibir todo por props. ☐ Pantalla Isleta: al tocar una celda se abre un panel con foto, último registro y botones "Cargar registro" y "Ver ficha". ☐ Alternar entre plano y lista, y buscador que resalta la celda.
Está listo cuando	Desde el celular se abre una isleta del seed, se busca la A-1-45 y se ve resaltada en el plano.
Requerimientos	ISL-01 a 06

Semana 2 · Cargar el mes

T13 · Fotos y guardar registro	Persona A (en pareja con B) · Semana 2
Qué es	Todo lo del servidor para guardar un registro con su foto. Es lo que usa la pantalla de B.
Backend (Go)	☐ Endpoint `POST /api/fotos`: recibe la imagen, la sube a R2 y guarda la ruta en la base. Devuelve el id. ☐ Endpoint `GET /api/fotos/{id}`: devuelve la imagen solo si el usuario puede ver ese lugar. ☐ Endpoint `POST /api/celulas/{id}/registros` con estas validaciones: permiso del lugar, foto obligatoria, fecha no futura ni anterior a la plantación, 4 individuos, y si el individuo está vivo: altura (1 a 999), cantidad de flores (0 a 100), presencia de frutos y nuevas plántulas obligatorias; 409 si ya hay registro de ese mes. ☐ Endpoint `PUT /api/registros/{id}` para editar, guardando quién lo modificó. ☐ Primer día: hacer una versión simple del POST que solo guarde, para que B pueda probar. Las validaciones se suman después.
Frontend (React)	☐ Componente Foto: botones cámara y galería, compresión con browser-image-compression (lado mayor 1600 px), barra de progreso y reintentar.
Está listo cuando	Con Postman o curl se guarda un registro válido; uno inválido devuelve el error correcto; un cargador no puede guardar en otro lugar.
Estado (18/10)	El PR #16 incluye el backend y el componente Foto, y se mergeó. Pendientes menores: probar el corte de señal a mitad de subida y mover `FotoRegistro` a components/ (lo coordinan A y C).
Requerimientos	CAR-04, 05 · RN-04 a 08 · AUD-01 · RNF-SEG-04

T14 · Pantalla de carga mensual	Persona B (en pareja con A) · Semana 2
Qué es	La pantalla más importante: cargar el mes de una célula, con la altura y las flores en desplegable y los individuos muertos en gris.
Backend (Go)	☐ Endpoint `GET /api/celulas/{id}/ultimo-registro`: devuelve la célula, el período en curso, el registro del mes en curso (si ya hay) y el último registro anterior, con sus individuos (formato en la sección 7). Va en `handlers/ultimo_registro.go`.
Frontend (React)	☐ Encabezado con código, lugar, isleta, período y botón "Cambiar" (abre el Plano de C para elegir otra). ☐ Foto del mes anterior al lado del componente Foto de A. ☐ 4 bloques (Árbol, Arbusto, Gramínea, Rastrera), cada uno con: BotonesVivoMuerto, altura en un desplegable de centímetros (select nativo, 1 a 999), cantidad de flores con SelectorNumero (0 a 100, arranca en 0), y presencia de frutos y nuevas plántulas con BotonesSiNo (arrancan en No). ☐ Al marcar Muerto, pedir confirmación; el bloque queda en gris, sin controles y con todo oculto. ☐ Si el individuo ya estaba muerto el mes anterior, el bloque aparece en gris con el botón "Cargar uno nuevo (reposición)"; al confirmar, se habilita como vivo. Ese botón no aparece en otros casos. ☐ Cada altura arranca con el valor del mes anterior (el desplegable se abre parado ahí) y muestra la diferencia. Si al guardar alguna no cambió, preguntar si es correcto. ☐ Observaciones (opcional). No van diámetro ni polinizadores: se descartaron el 16/10. ☐ Si ya hay registro del mes (o el POST responde 409 con `registroId`), abrir en modo edición. ☐ Al guardar: mensaje de éxito y botón "Siguiente célula pendiente".
Ojo	El backend de T13 acepta que un individuo pase de Muerto a Vivo, así que la reposición no necesita cambios en Go. Mientras el PR #16 no esté mergeado, el guardado se prueba con un mock (regla 5).
Está listo cuando	Desde el celular se carga un registro completo con foto y aparece al día en la isleta. Se midió cuánto tarda y se anotaron los pasos lentos (la clienta aceptó que pase de 2 minutos).
Requerimientos	CAR-01 a 10, 13 a 15, 18 a 20 (CAR-11 y CAR-12 descartados) · RN-08, RN-09

T15 · Ficha de la célula	Persona C · Semana 2
Qué es	La historia completa de una célula.
Backend (Go)	☐ Endpoint `GET /api/celulas/{id}` con datos, 4 individuos y todos los registros ordenados por fecha.
Frontend (React)	☐ Encabezado con migas de pan, código, fecha de plantación y estado. ☐ 4 tarjetas de individuos con estado, altura y crecimiento desde el registro inicial. El individuo muerto se ve en gris. ☐ Gráfico con Recharts y selector de variable (altura o cantidad de flores): una línea por forma de vida. ☐ Línea de tiempo de fotos con fecha y alturas; al tocar, la foto se ve grande. ☐ Botón "Editar" en cada mes, solo si puede editar (lleva a la carga de B en modo edición).
Está listo cuando	Se abre la ficha de una célula del seed y se ven los 3 meses en el gráfico y en las fotos.
Requerimientos	FIC-01 a 05

Semana 3 · Versión de prueba

T16 · Usuarios	Persona A · Semana 3
Qué es	Que el administrador cree y maneje usuarios.
Backend (Go)	☐ Endpoints GET, POST y PUT `/api/usuarios`, solo para admin. ☐ Al crear un usuario, generar un enlace de invitación de un solo uso (vence en 72 h) para que defina su contraseña. Simplificación: el admin copia el enlace y lo manda por WhatsApp; el email automático queda para después. ☐ Un solo cargador por lugar; no se puede deshabilitar al último admin. ☐ Tests de permisos: cargador guardando en otro lugar, lector intentando guardar, usuario sin sesión.
Frontend (React)	☐ Pantalla Usuarios: lista con filtros, formulario de alta y edición, deshabilitar y generar nuevo enlace. ☐ Pantalla para definir la contraseña desde el enlace.
Está listo cuando	El admin crea un cargador, le pasa el enlace, el cargador define su contraseña y entra; los tests de permisos pasan.
Requerimientos	USR-01 a 07 · AUT-04, 05 (simplificados) · RNF-SEG-01

T17 · Nueva célula, editar registro y borrador	Persona B · Semana 3
Qué es	Dar de alta células desde el celular y no perder lo cargado si se corta la señal.
Backend (Go)	☐ Endpoint `POST /api/celulas`: valida número y celda libres y crea los 4 individuos.
Frontend (React)	☐ Pantalla Nueva célula: elegir isleta, número (sugerir el siguiente libre) y celda libre en el Plano; mostrar el código grande. ☐ Al confirmar, seguir con el registro inicial reutilizando la pantalla de carga. ☐ Editar un registro desde la ficha (modo edición de la carga). ☐ Borrador: mientras se completa la carga, guardar lo cargado en el celular (localStorage) y ofrecer retomarlo si se cierra la app.
Está listo cuando	Se da de alta la célula A-1-61, se carga su registro inicial y aparece en el plano; si se cierra la app a mitad de carga, al volver se ofrece retomarla.
Requerimientos	ALT-01 a 06, 09 · FIC-05 · CAR-16

T18 · Resultados y métricas	Persona C · Semana 3
Qué es	La pantalla que muestra el impacto con pocas métricas claras.
Backend (Go)	☐ Endpoint `GET /api/metricas` con filtros de lugar, isleta, período y forma de vida. Calcula con SQL: células plantadas, tasa de supervivencia, crecimiento promedio, individuos en floración, con frutos y células con nuevas plántulas, y los datos de los gráficos (fórmulas en la sección 7 del documento v5). ☐ Tests con el seed: calcular a mano un caso y comparar.
Frontend (React)	☐ Pantalla Resultados: filtros arriba, 3 o 4 tarjetas de métricas, supervivencia por forma de vida con su explicación, evolución de altura y de supervivencia. ☐ Mensaje cuando todavía no hay datos suficientes.
Ojo	Un individuo repuesto mezcla la planta vieja con la nueva en el crecimiento promedio. Hasta que la clienta responda la pregunta 19 del v5, se calcula como dice la sección 7. No hay métrica de polinizadores (descartada).
Está listo cuando	Al cambiar un filtro cambian todas las métricas y coinciden con el cálculo a mano.
Requerimientos	RES-01 a 05, 07 · Sección 7 del documento v5

T19 · Versión de prueba para la clienta	Todos · Jueves 22 o viernes 23/10
Qué es	Mostrarle la app a la clienta y anotar lo que pida.
Está listo cuando	Se hizo la demo, las observaciones están anotadas como tickets nuevos en el tablero.

Semana 4 · Campo y público

T20 · Lugares, isletas, planos y backups	Persona A · Semana 4
Qué es	Que el admin cargue los lugares reales, y que los datos estén protegidos.
Backend (Go)	☐ Endpoints de `/api/admin/lugares` e `/api/admin/isletas` (crear y editar). ☐ Endpoint para subir el plano (imagen a R2, filas y columnas). ☐ Endpoint para crear células en lote (de un número a otro, ubicándolas en celdas libres). ☐ Resolver los backups diarios de la base: la base gratuita de Render no tiene backups y se borra a los 30 días, así que hay que pasar a un plan pago (a acordar con UCC, pregunta 13) o programar un `pg_dump` diario a R2. Hacer también una copia semanal de las fotos. Probar restaurar un backup.
Frontend (React)	☐ Pantallas de Lugares e Isletas con formulario, subida del plano y vista previa de la grilla (reutiliza el Plano de C).
Está listo cuando	Se carga un lugar nuevo con su isleta, plano y 20 células desde la app; se restauró un backup en una base de prueba.
Requerimientos	LUG-01 a 08 · RNF-DIS-01 a 03

T21 · Tabla de datos y Excel	Persona B · Semana 4
Qué es	La tabla o matriz que pidió la clienta, con descarga a Excel.
Backend (Go)	☐ Endpoint `GET /api/datos` paginado (50 filas) con los filtros de resultados. ☐ Endpoint `GET /api/datos/excel` que arma el archivo con excelize, con las columnas de la sección 12 del documento v5 (sin diámetro ni polinizadores, que se descartaron el 16/10).
Frontend (React)	☐ Pantalla Datos: tabla con la columna de código fija y desplazamiento horizontal en el celular. ☐ Interruptor para ver como matriz (células en filas, meses en columnas). ☐ Filtros, buscador y botón "Descargar Excel".
Está listo cuando	Se filtra por un lugar, se descarga el Excel y abre bien en Excel con acentos y decimales.
Requerimientos	TAB-01 a 05 · Sección 12 del documento v5

T22 · Página pública y QR	Persona C · Semana 4
Qué es	Lo que ve la gente que escanea el QR el día de la plantación.
Backend (Go)	☐ Endpoint `GET /api/publico/{letra}` sin sesión: solo lugares marcados como públicos, sin nombres de usuarios. Reutiliza el cálculo de métricas de T18.
Frontend (React)	☐ Pantalla pública liviana (sin menú ni barra de carga): explicación de qué es una célula de bosque, métricas y gráficos. ☐ Pantalla solo para admin que genera el QR del Campus UCC y descarga el cartel en PDF o PNG.
Está listo cuando	Se escanea el QR con un celular con datos móviles y la página abre en menos de 3 segundos.
Requerimientos	PUB-01, 02, 03, 07 · QR-01, 02 · RNF-REN-01 · PRI-02

T23 · Prueba en campo y datos reales	Todos · Semana 4
Qué es	Probar la app donde se va a usar y cargar los datos de verdad.
Está listo cuando	Un cargador real cargó varias células en el lote, se midió el tiempo (la clienta aceptó que pase de 2 minutos) y se anotaron los pasos lentos, se cargaron los lugares, isletas, planos y usuarios reales, y los problemas quedaron como tickets.
Requerimientos	CAR-17

Semana 5 · Lanzamiento

T24 · Correcciones	Todos · Semana 5
Qué es	Resolver lo que salió de la prueba en campo y de la clienta, empezando por lo que impide cargar.
Está listo cuando	No queda ningún problema que impida cargar un registro o ver la página pública.

T25 · Manual para cargadores	Persona B · Semana 5
Qué es	Una hoja con capturas: encontrar la célula en el plano, sacar la foto, cargar el mes y qué hacer si un individuo murió o se repuso.
Está listo cuando	Un docente que nunca vio la app carga un registro solo con la hoja.

T26 · Demo de la plantación	Persona C · Semana 5
Qué es	Guion corto, QR impreso y recorrido para presentar la app el día de la plantación.
Está listo cuando	Se ensayó la demo entre los tres.

T27 · Documento de traspaso	Persona A · Semana 5
Qué es	Todo lo que necesita quien siga: cuentas y contraseñas de servicios (entregadas en persona), costos, cómo publicar, cómo agregar un lugar o una variable y cómo restaurar un backup.
Está listo cuando	UCC Sostenible recibió el documento y tiene acceso a todas las cuentas.
Requerimientos	RNF-MAN-03, 04

Cambio transversal · Gramínea

T28 · Cambiar Herbácea por Gramínea	Persona B · Después del PR #16
Qué es	La clienta aclaró el 16/10 que la tercera forma de vida es Gramínea y no Herbácea. Hay que cambiar el nombre en todo el proyecto. El dígito 3 y los ids no cambian.
Pasos	☐ Avisar en el grupo antes de empezar: hay una migración nueva (regla 3) y se tocan archivos de las tres. ☐ Esperar a que se mergee el PR #16 (T13) y crear la rama (por ejemplo `cambio-gramineas`) desde main actualizado. ☐ Buscar todas las apariciones de herbácea, herbáceas y herbacea (con o sin tilde, en singular y plural) en backend, frontend, seed, migraciones, tests, README, los CLAUDE.md y la carpeta diseno/. ☐ Crear una migración nueva de goose que cambie el nombre en la tabla de formas de vida, con su Down. No se editan las migraciones que ya corrieron. ☐ Actualizar el seed, el mapa de nombres de formas de vida de registros_validacion.go, los componentes, los textos de la interfaz, los tests y los CLAUDE.md, cuidando el género y el plural ("una gramínea", "las gramíneas"). ☐ Correr `go vet`, `go test` y el build del frontend, y volver a buscar para confirmar que no quedó ninguna. ☐ Cambiar el nombre también en las pantallas de Stitch.
Está listo cuando	Una búsqueda de "herb" en el código no encuentra nada, los tests pasan y en el celular se ve Gramínea en la carga, la ficha, los gráficos y la tabla de datos.
Requerimientos	Sección 2 y RNF-USA-06 del documento v5

9. Qué hacer si te trabás

Tiempo trabado	Qué hacer
30 minutos	Releé el error completo, buscalo en internet y probá con un ejemplo más chico. Preguntale a Claude pegando el error y el código.
1 hora	Escribí en el grupo: qué querés hacer, qué probaste y el error exacto. No es molestar: es parte del trabajo.
2 horas	Sesión en pareja con quien esté más libre, compartiendo pantalla.
Medio día	Se habla en la reunión: se simplifica el ticket, se lo parte en dos o lo toma otra persona.

Para aprender rápido

Go: el tour oficial (go.dev/tour) y "Go by Example" (gobyexample.com). React: la documentación oficial en react.dev, sección "Learn". SQL: cualquier tutorial de PostgreSQL de SELECT, JOIN y GROUP BY. Cuando uses una herramienta nueva, primero hacé funcionar su ejemplo más simple y después adaptalo al proyecto.

10. Lo que queda para después

Estas funciones están en el documento de requerimientos pero no entran en las 6 semanas. Quedan en el tablero en una columna "Después" y se hacen solo si sobra tiempo o las toma quien siga.

Función	Requerimientos	Nota
Mapa de lugares	MAP-01 a 04	Prioridad media.
Emails automáticos de invitación y recuperación	AUT-04, 05	Mientras tanto, el admin copia y manda el enlace (T16).
Recordatorios de carga	NOT-01 a 05	Prioridad baja.
Historial de cambios de cada registro	FIC-06, AUD-02	Ya se guarda quién modificó (T13).
Baja de célula y anulación de registro	LUG-09, FIC-08	Mientras tanto, lo hace el admin en la base.
Descargar fotos en ZIP	TAB-06	—
Galería pública y compartir por WhatsApp	PUB-04, 05	—
Comparar fotos antes y después	FIC-07	—
Límite de intentos de login	AUT-10	—
Marca de individuo repuesto y crecimiento medido desde la reposición	Pregunta 19 del v5	Solo si la clienta lo confirma. Toca el catálogo de variables, el guardado de T13 y la métrica de T18.
Carga sin conexión con sincronización	RNF-OFF-01	Si la clienta confirma que no hay señal en los lotes, pasa a ser prioritaria y la hace B en la semana 4 sobre el borrador de T17; la tabla de datos (T21) se reparte entre A y C.